Highlights

Proof-Carrying Contracts for Task-Preserving Power-System Model Transformation

Zixuan Liu, Wei Xiong

- Proof-carrying contracts bind conversion evidence to the declared grid task.
- Full PCC rejected 1,320 harmful transformations and accepted 660 controls.
- PCC prevented every consequential N-1, AC-OPF, and DC-SCOPF solver start tested.
- PCC blocked all 369 false-secure dispatches in the 50-state DC-SCOPF campaign.
- Solver and CGMES controls confirmed portability across tools and data sources.

Proof-Carrying Contracts for Task-Preserving Power-System Model Transformation

Zixuan Liu[a], Wei Xiong[b,c,*]

[a]*Detroit Green Technology Institute, Hubei University of Technology, , Wuhan, 430068, , China*
[b]*School of Electrical and Electronic Engineering, Hubei University of Technology, , Wuhan, 430068, , China*
[c]*Department of Computer Science and Engineering, University of South Carolina, , Columbia, SC 29201, , USA*

Abstract

Power-system studies often pass network models through several conversions before contingency analysis or optimal power flow. A converted model can still be syntactically valid and solvable even when the conversion changes an asset, relation, parameter, or intervention needed by the study. This paper proposes proof-carrying contracts (PCC) to check that the converted model still represents the intended task before it is sent to a solver. The contract links source and target snapshots with authoritative asset relations and task-specific requirements. We evaluated PCC on 22 public networks and 1,980 transformations. It rejected all 1,320 harmful transformations and accepted all 660 lawful transformations. Simpler checks based on signatures, identity, task coverage, or attributes each missed at least one failure class. PCC also stopped all consequential cases in the AC N-1 and AC-OPF studies. For DC-SCOPF, it blocked 369 false-secure dispatches found across 50 operating states and 12,340 candidate outages. The corresponding median effects were 3.879 percentage points in N-1 loading, 5.96% in AC-OPF cost, and 0.241 p.u. of DC-SCOPF loading excess. The main results were reproduced with PYPOWER/PIPS and Julia/PowerModels/HiGHS, and tests with CGMES data gave consistent decisions. These results show that PCC can prevent model-conversion errors from reaching power-system solvers.

*Corresponding author
Email address: xw@mail.hbut.edu.cn (Wei Xiong)

Keywords: proof-carrying contract, power-system model transformation, model validation, CGMES, security-constrained optimal power flow, contingency analysis

1. Introduction

Contingency analysis and optimal power flow rarely operate directly on an exchanged network model. A Common Information Model (CIM) or Common Grid Model Exchange Specification (CGMES) package is parsed, normalized, converted into a tool-specific network, and mapped to solver variables before the study begins [4, 8, 12]. The same model may pass through software environments that connect data exchange, network analysis, and optimization. Security-constrained formulations and learned solution methods continue to improve the speed and scale of the final computation [1, 2]. Their results are useful only if the solver receives the network and task that the analyst intended to study.

Errors introduced along this chain are not necessarily visible at the solver interface. A converted model may satisfy its schema, import successfully, and converge after a branch has been omitted, two independent devices have been merged, a limit has changed, or an intervention has been mapped to the wrong asset. In an N-1 study, a missing outage target or an altered rating can lower the reported loading and turn an insecure case into an apparently secure one. In optimal power flow, a changed generator limit, cost, or bus assignment can alter feasibility and dispatch cost. The numerical solution can therefore be correct for the converted model while answering a different operational question.

Current validation mechanisms cover important parts of this process. CIM and CGMES profiles define exchange structures, while SHACL-based approaches check graph constraints and model rules [5–7]. Signatures protect artifacts, identifiers match objects, import tests establish compatibility, and solver checks establish numerical feasibility. None of these checks alone shows that the assets, relations, parameters, and interventions required by a study have survived the complete transformation. That joint requirement is the central problem addressed here.

This paper proposes proof-carrying contracts (PCC) for task-preserving power-system model transformation. A PCC binds source and target snapshots to authoritative asset relations, task-selected equipment, required attributes and tolerances, and the interventions applied by the study. The

verifier checks this evidence before computation and sends the converted model to the solver only when the declared task has been preserved. PCC thus places task preservation directly in the execution path between model conversion and power-system analysis.

The main contributions are fourfold.

First, we formalize task preservation as a proof obligation over source and target snapshots, authoritative asset relations, required attributes, intervention semantics, and conversion evidence, and implement the obligation as a signed contract and solver-release decision.

Second, we develop verification rules for identity, task coverage, relations, attributes, trace consistency, and interventions. An ordered baseline ladder isolates the contribution of each evidence class and shows why the complete contract is required.

Third, we connect model-transformation errors to physical and economic consequences in AC N-1, AC-OPF, and DC-SCOPF studies, and measure the harmful computations stopped before solver execution.

Fourth, we demonstrate portability with official CGMES material, an untouched public CGMES source, an external-tool roundtrip, and independent PYPOWER and PowerModels implementations.

The evaluation covers 22 public networks and 1,980 transformations. PCC rejected all 1,320 harmful transformations and accepted all 660 lawful transformations. It stopped every consequential case found in the AC studies and blocked all 369 false-secure dispatches in the 50-state DC-SCOPF campaign. The hidden errors changed branch loading, operating cost, and load shedding, showing that task preservation affects power-system conclusions rather than only model metadata.

The remainder of the paper reviews related work, formulates task preservation, presents the PCC method and evaluation design, reports the experimental results, and discusses their implications for power-system model transformation.

2. Related Work

2.1. Power-system model conversion and validation

Power-system analysis programs generally do not work directly with exchange files. They first convert the files into internal objects for buses, branches, generators, loads, topology, and operating states. Data-management packages provide these objects in a form that can be used by

network-analysis and optimization routines [4]. CIM and CGMES define the information models and application profiles used to exchange the underlying grid data between organizations and software tools [8, 9, 12].

Validation at this layer mainly concerns structure and model rules. SHACL can express graph, cardinality, value, and profile constraints, and has been used with CGMES and power-system knowledge graphs [5–7, 10, 11]. These checks show whether an exchange artifact conforms to its declared requirements. An import test answers a separate question: whether a given tool can construct a network from that artifact. PCC checks what happens after conversion. It tests whether the internal network still contains the equipment, relations, parameters, and interventions needed for the study.

This distinction is important because the same exchange-valid object can support different operational meanings after it enters a modelling stack. A topology processor may collapse a switching representation, a converter may translate terminals into branch endpoints, an importer may assign local identifiers, and an optimization wrapper may select the variables needed by a study. PCC adds the missing execution-layer condition: before a numerical study is launched, the transformed object must carry checkable evidence for the assets and actions that define the source-model task. It builds on CIM, CGMES, SHACL, and import testing by converting structural validity into task-valid solver release.

2.2. Security assessment and optimization

N-1 analysis, OPF, and SCOPF start from a specified network, operating state, and set of contingencies. Recent work has improved large-scale formulations, contingency filtering, uncertainty treatment, and learned approximations [1–3]. Performance is normally assessed through feasibility, objective value, security violations, and solution time.

These methods assume that model preparation has already produced the intended problem. If conversion changes a selected branch, generator limit, network relation, or outage action, the solver may still return a feasible and optimal result for the model it received. PCC is placed before that computation. It checks that an N-1, AC-OPF, or DC-SCOPF instance still represents the task selected on the source model; the numerical method can then solve the verified instance in the usual way.

This placement separates PCC from solver verification. OPF and SCOPF methods evaluate a formulation, approximation, or numerical procedure for

a given mathematical model. PCC verifies that the mathematical model entering the solver still represents the declared grid task. A strong optimizer can produce a precise answer to a changed problem when a conversion removes the contingency target, changes a branch limit, or shifts a generator cost. PCC makes task preservation explicit before that computation begins.

2.3. Proof-carrying and provenance-based execution

Proof-carrying code provides a useful precedent for checking machine-readable evidence before execution [13]. Provenance models record the entities, activities, and derivation relations involved in producing a computational artifact [18–20]. An artifact can therefore be accompanied by both its processing history and evidence that can be checked automatically.

For power-system model conversion, that evidence must refer to the study being run. PCC records the selected equipment, authoritative source-to-target relations, required parameters, and intervention semantics in the contract. The verifier uses those records when deciding whether the converted network can enter contingency analysis or optimization. A provenance record describes how the artifact was produced, and a signature protects its contents; PCC uses both but adds the task information needed to decide whether the model is suitable for the requested calculation.

The closest software-engineering relatives are design by contract, proof-carrying authorization, and assume-guarantee contract frameworks [14–17]. Those approaches use explicit obligations to control whether a component may be used, whether an action is authorized, or whether a composed system satisfies an interface assumption. PCC adapts that idea to grid-model transformation, where the controlled action is not a method call or an access request but the release of a transformed network to a power-system solver. Its obligations are therefore stated in grid terms: task-selected assets must be covered, relation cardinalities must be allowed, required electrical and economic attributes must remain within tolerance, and intervention targets must still refer to the intended operational objects.

This framing creates a new role for contracts in power-system computation. The contract is not used only to document an exchange or to annotate a workflow. It becomes an executable condition for a protected solver call. The contribution is therefore both semantic and operational: PCC states what must be preserved for a declared study, and the gate enforces that statement before a numerical result can be produced. This moves model-transformation assurance from retrospective validation to controlled execution.

3. Problem Formulation

Each study considered here starts from a source network snapshot and runs on a transformed, solver-facing snapshot. Let S denote the source snapshot, T the transformed target snapshot, and τ the operational task. The task is an N-1 contingency assessment, an AC-OPF calculation, or a DC-SCOPF calculation. It selects source assets A_S^τ, target assets A_T^τ, required attributes K^τ, attribute tolerances ϵ^τ, and intervention semantics I^τ. These sets cover the buses, branches, generators, ratings, limits, costs, terminal relations, and outage or dispatch actions needed by the study.

A model transformation is therefore evaluated at the level used by the study, not only as a whole-file conversion. It must show how task-relevant equipment in S corresponds to task-relevant equipment in T. The converter provides a relation set

$$R = \{(U_i, V_i, r_i, e_i)\}_{i=1}^{m},$$

where $U_i \subseteq A_S^\tau$ and $V_i \subseteq A_T^\tau$ are source and target asset sets, r_i is the declared relation type, and e_i is evidence from the converter trace.

A transformed target is task-safe for τ when it satisfies the following conditions:

1. the source and target task projections match the hashes recorded in the certificate;
2. the issuer, signature, timestamp fields, certificate identifier, transformation identifier, and nonce are valid;
3. every source and target asset selected by τ is covered by an allowed relation;
4. the declared relation cardinality and identity constraints are satisfied;
5. every required operational attribute is preserved within ϵ^τ;
6. the intervention map preserves the requested outage, dispatch, or observation action;
7. the authoritative converter trace agrees with the certificate relation set.

The verifier returns `accept`, `reject`, or `unresolved`. The execution gate starts the N-1, OPF, or SCOPF solver after an `accept` decision. A `reject` decision identifies a demonstrated task mismatch. An `unresolved` decision identifies the evidence needed to complete the task decision. The gate also

emits a receipt binding the verified hashes, task, decision, reasons, and solver-start status.

This formulation separates structural validity from task preservation. A target network can be parseable and solvable while changing the operational study. PCC releases the target after the study equipment, parameters, relations, and interventions are linked to their source-model counterparts.

PCC gives a task-bound execution guarantee. The source task declaration specifies the intended study, the converter trace records the authoritative source-to-target relations, issuer keys identify the approved model-preparation path, and the protected gate controls solver access. With these inputs, an accepted target satisfies the stated task-preservation obligations before execution. A new task receives its own task projection and, when selected assets or interventions change, its own certificate.

This task locality matches operational practice because most computations use a defined subset of the full exchange model. A contingency assessment depends on a branch, its terminal buses, ratings, and outage action. An OPF study also depends on generator limits, costs, loads, and bus assignments. A DC-SCOPF study depends on the base dispatch, candidate outages, post-contingency flow model, monitored limits, and load-shedding variables used to define feasibility. PCC records the subset that the task uses and makes that subset the unit of solver release.

3.1. Task-semantic corruption model

PCC targets the conversion stage where a parseable target model can diverge from the operational task selected on the source model. The verifier uses a declared task contract, a source-snapshot registry, authorized issuer keys, an authoritative converter trace, and a protected route to the downstream solver. Together, these inputs let the gate check whether the transformed network still represents the selected study.

Task-semantic corruption denotes a conversion that changes the solver-facing task while leaving import or convergence intact. Examples include omitting a selected branch, merging two independent devices, reusing the wrong target identifier, changing a required rating or generator limit, or mapping an outage action to the wrong asset. An accepted certificate establishes that the transformed target satisfies the declared snapshot, relation, coverage, attribute, intervention, trace, freshness, and issuer obligations before solver execution.

The corruption model separates missing evidence from demonstrated mismatch. A missing relation for a selected source asset produces `unresolved` and returns the evidence needed for repair. A demonstrated mismatch, such as duplicate target reuse or an intervention mapped to the wrong branch, produces `reject`. This decision structure supports both prevention and repair: unique authoritative evidence completes a certificate, while ambiguous evidence keeps solver release closed.

4. Method

PCC sits at the handoff between model conversion and power-system computation. It takes the source snapshot, the transformed target snapshot, the declared operational task, and relation evidence from the converter. It then returns a release decision for the downstream N-1, OPF, or SCOPF solver. A signed certificate binds the source and target task projections to the declared task, and the verifier checks the relations, attributes, and interventions before the solver is called.

Figure 1 summarizes the task-semantic contract, verification workflow, and fail-closed execution logic.

4.1. Certificate data model

The PCC certificate carries the information used by the release gate. It stores the source and target task-projection hashes, task contract, relation records, authoritative trace digest, transformation and certificate identifiers, issuer, issue time, nonce, and Ed25519 signature. The task contract identifies the assets, attributes, tolerances, and interventions required by the downstream study.

The certificate fields align with the notation in Section 3. The source and target hashes bind the task projections of S and T. The task identifier binds the certificate to τ. The relation records bind subsets of A_S^τ to subsets of A_T^τ, with a declared relation type and trace evidence. The attribute records bind each required quantity in K^τ to its tolerance in ϵ^τ. The intervention records bind outage, dispatch, or observation actions in I^τ to the corresponding target elements.

Before signing, the snapshot hashes, task identifier, and relation list are canonicalized so that the same task projection gives the same digest. The signature protects the certificate contents, the nonce and identifiers prevent replay and cross-task reuse, and the issuer fields bind the evidence to an

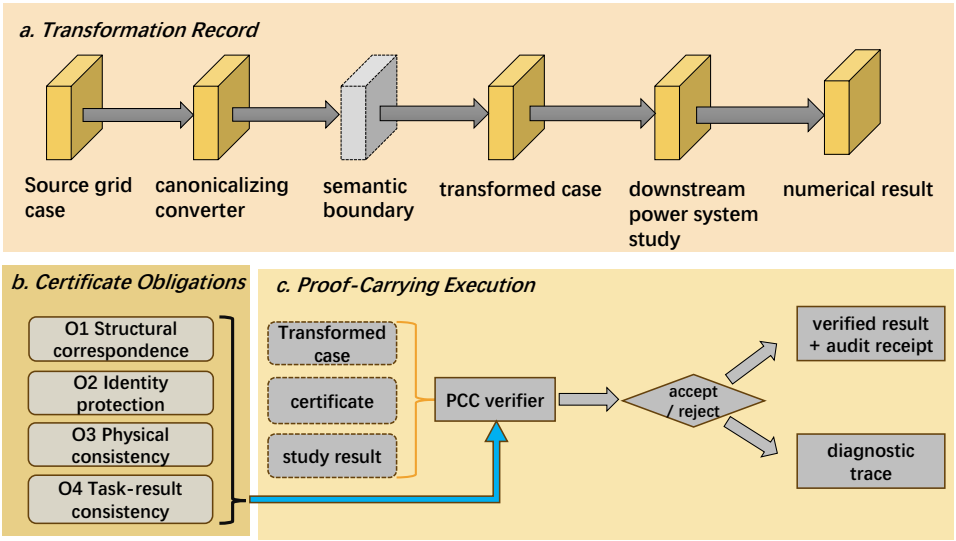

Figure 1: Task-semantic proof-carrying contract and fail-closed execution. A transformed model reaches the protected solver after a signed, task-bound contract verifies the snapshots, relations, attributes, trace, and intervention semantics.

authorized conversion path. The same record therefore carries both the conversion evidence and the solver-release conditions.

The certificate is a compact task-evidence object. It contains the evidence needed to decide whether the declared task may run. For each task asset, the relation record stores the source identifier, target identifier or target set, relation type, and trace reference. For each required attribute, the record stores the source value, target value, tolerance, unit or categorical domain, and comparison rule. For each intervention, the record stores the source action, target action, selected object, and the relation that justifies the mapping. These fields give the verifier a normalized evidence source. The gate checks the referenced task projection and emits a machine-readable decision reproducible from the archived inputs.

Canonicalization keeps the evidence stable across legitimate representational differences. The implementation sorts relation records by canonical source and target keys, serializes numerical values with fixed comparison rules, and hashes the task projection. This design makes lawful renames possible in the experiments. A target can use different local identifiers and pass when the certificate gives an authorized relation and the task attributes

and interventions remain preserved. The release decision therefore follows task evidence rather than local naming.

4.2. Task-preservation checks

The verifier first confirms that every task-selected source and target asset is represented by an allowed relation. Allowed relations include exact one-to-one identity and explicitly authorized aggregate or split relations. This step keeps selected branches, generators, buses, and contingency targets visible across conversion.

The verifier then checks relation cardinality, duplicate target reuse, independent merges, one-to-many intervention semantics, attribute preservation, and trace agreement. Attribute checks apply only to the quantities required by τ, such as branch limits, generator bounds, costs, bus assignments, endpoint parameters, or status fields. Intervention checks ensure that the requested outage, dispatch change, or observation remains attached to the corresponding target element. Trace agreement verifies that the certificate relation set is supported by the authoritative converter record.

The checks are evaluated in a fixed order. Identity and snapshot checks bind the certificate to the requested task. Coverage checks verify that the task asset set is present. Relation checks test cardinality and target reuse. Attribute checks test the required numerical or categorical quantities. Intervention checks test whether the requested action is applied to the same operational object. The decision records which coverage, identity, relation, attribute, intervention, or trace condition controls solver release.

The fixed order is part of the audit design. Early checks reject stale certificates, wrong snapshots, unauthorized issuers, and cross-task reuse before any semantic inference is made. Coverage checks then establish that the task footprint is present on both sides of the transformation. Relation checks distinguish lawful one-to-one, split, and aggregate mappings from independent merges or duplicate target reuse. Attribute checks are task-dependent rather than global: a branch thermal limit matters for contingency loading, a generator cost matters for OPF, and a bus-voltage parameter matters only when it is selected by the task contract. Intervention checks are last because they combine the relation and attribute evidence with an action, such as removing a line, changing a dispatch variable, or observing a post-contingency flow.

This order produces decision reasons that are useful to a model engineer. A `task_target_missing` reason points to an absent target object. A `relation_cardinality_invalid` reason points to a

mapping that must be split, aggregated, or rejected explicitly. An `attribute_mismatch` reason points to the quantity and tolerance that failed. A `task_selector_not_preserved` reason points to an outage or dispatch action that no longer selects the intended target. The same taxonomy is used in the experiments so that prevention results and repair diagnostics come from one verifier rather than from a separate post-hoc analysis.

4.3. Solver release rule

Verification and solver start occur in one gate call. The gate returns `accept`, `reject`, or `unresolved`. An `accept` decision releases the transformed model to the N-1, OPF, or SCOPF solver. The gate writes a receipt with the verified hashes, decision, reasons, latency, and solver-start status.

The gate follows six steps. First, it canonicalizes the source and target task projections and computes their hashes. Second, it verifies issuer authorization, signature, identifiers, time fields, and nonce. Third, it binds the certificate to the requested task and snapshots. Fourth, it checks relation cardinality, task-asset coverage, required attributes, intervention preservation, and agreement with the authoritative converter trace. Fifth, it returns `accept`, `reject`, or `unresolved` with machine-readable reasons. Sixth, it invokes the solver for accepted targets and writes the receipt.

`accept` means that the task contract is satisfied for the requested source, target, and solver task. `reject` identifies a verified task-preservation violation. `unresolved` identifies the authority or trace evidence needed to complete the task decision. The reason field points to the transformation element to update before issuing a new certificate.

The solver-release rule is fail-closed. Only `accept` invokes the downstream solver. Both `reject` and `unresolved` produce zero protected solver starts and write an audit receipt. This rule turns task evidence into an execution precondition. PCC therefore operates as a release gate: it checks the requested solver call against the supplied evidence and records the decision at the computation boundary.

The receipt gives the release decision a reproducible audit trail. It binds the source and target task hashes, the verified certificate, the task identifier, the decision reasons, the verifier latency, and the solver-start status. The receipt is useful after both accepted and blocked runs. For accepted runs, it records the evidence that authorized the computation. For blocked runs, it records the exact obligation that stopped execution. This makes the solver interface accountable without changing the downstream numerical algorithm.

4.4. Comparison methods and transformation scenarios

We compare six ordered checks:

- B0: structural availability;
- B1: signed artifact binding;
- B2: global identity coverage;
- B3: task-footprint coverage;
- B4: task-attribute invariants;
- B5: full PCC with signed snapshot, relation, trace, and intervention binding.

The ladder is evaluated on identical paired transformations so that adjacent improvements can be tested by exact McNemar tests. Holm correction controls familywise error across adjacent comparisons. The order matters because each level adds a distinct task-preservation capability. Signing binds artifact identity, global identity prevents cross-artifact reuse, task-footprint coverage ensures that the selected assets are present, attribute invariants catch silent drift in required quantities, and full PCC binds relation, trace, and intervention evidence to the solver-release decision. This keeps the comparison additive.

Each baseline adds one semantic obligation, making the contribution of identity, coverage, attributes, and authoritative relations directly measurable.

The baselines represent checks that are common or plausible in production pipelines: a file is available and importable, a signed artifact is present, global identifiers appear to match, task-selected objects are listed, or task attributes agree within tolerance. The experiment measures the protection added by each obligation. The ordered design makes this contribution visible because every harmful scenario is evaluated under the same source, target, and task declaration while the release obligation changes.

The semantic campaign uses 22 public networks. Each network contributes 30 lawful authoritative renames and 60 harmful transformation scenarios: task-asset drop, independent merge, incorrect one-to-many mapping, target-identifier reuse, endpoint-parameter swap, and source-snapshot mismatch. These scenarios represent conversions that leave a target network

parseable while changing a branch, generator, relation, endpoint, intervention, or source snapshot needed by the operational task. The main endpoints are lawful acceptance, harmful acceptance, and the one-sided Clopper-Pearson upper confidence bound when no harmful event is accepted. The six families isolate coverage, identity, relation cardinality, target reuse, intervention semantics, and snapshot binding. Lawful renames preserve every contract field and serve as positive controls.

Together, the six scenarios exercise the principal ways in which a model conversion can preserve syntax while changing a solver-facing task.

4.5. Operational studies and statistical analysis

The AC N-1 experiment compares the contingency conclusion obtained from the intended model with the conclusion obtained after a task-semantic transformation. The main effect is the counterfactual difference in maximum loading. The AC-OPF experiment records paired convergence, feasibility, objective value, and relative cost effect. Attempted and paired-valid denominators report solver availability and operational effect separately.

The DC-SCOPF campaign uses five networks, ten predefined operating states per network, every non-islanding line and transformer candidate, and post-contingency linear power flow. A false-secure dispatch passes the transformed baseline but violates the intended contingency assessment. The main endpoint is whether PCC stops that dispatch before solver execution.

The DC-SCOPF design evaluates both the release decision and the physical or economic consequence associated with each false-secure dispatch.

A strict false-secure row requires a matched full-model and transformed-model solution, a secure result under the transformed baseline, and a violation when the intended contingency is evaluated. Rows that fail solver pairing, exacerbate an already overloaded baseline, or belong to the broader legacy alias-overlimit label are retained in the archive as separate categories. This endpoint isolates the case that matters operationally: the transformed task would have allowed a dispatch that the intended task rejects.

Network is the confirmatory unit. We report exact attempted and paired-valid denominators, network-cluster bootstrap confidence intervals for median effects, and one-sided exact sign tests over network medians. Paired semantic comparisons use exact McNemar tests with Holm correction. Zero-event rates are accompanied by one-sided exact binomial upper bounds.

The statistical design is paired at the transformation level and clustered at the network level. Pairing removes differences caused by the base case, solver,

or operating state. Network-level summaries distribute the evidence across systems rather than allowing the largest case to dominate. Exact tests match the bounded count structure and the zero harmful protected-start outcome.

4.6. Portability and solver configuration

The standards comparison uses official ENTSO-E APL 1.1.1 shapes and CAS 3.0.3 material. QoCDC 4.1.4 evaluation covers 15 implemented Level 1 to Level 4 checks. Structural conformance and PCC decisions are reported as complementary outcomes.

For the untouched holdout, a PowSyBl core commit was fixed before tree inspection. A filename- and hash-based rule selected the first complete, nonduplicate CGMES bundle. The evaluation combines PCC, APL SHACL, and native import outcomes.

The independent comparison uses PYPOWER/PIPS on Windows and PowerModels/HiGHS under WSL Julia. The PowerModels DCP formulation provides a contrast through its transformer treatment, and the aligned DCMP formulation matches the PYPOWER `makeBdc` equations. This comparison separates formulation effects from solver implementation effects.

The final evaluation configuration fixed the networks, operating states, candidate outages, costs, equations, task transformations, safety thresholds, and termination rules before execution. Each network-state ran in a separate process with a terminal checkpoint. For case500, tighter interior-point tolerances reduced dual noise from 3.43×10^{-6} to 3.87×10^{-7} and isolated 15 active candidates at offset 7. The final configuration evaluated every selected candidate with the same task definition and acceptance logic.

5. Experimental Results

In all three operational studies, corrupted transformations remained parseable and solvable but changed the study result. PCC stopped the affected targets before solver execution in the AC N-1, AC-OPF, and DC-SCOPF campaigns. The ablation and portability tests then show which evidence was responsible for these decisions.

5.1. AC N-1 and AC-OPF

Corrupted transformations changed both contingency loading and OPF cost in the AC studies. The AC N-1 campaign produced 53 paired-valid outcomes from 56 attempts. PCC prevented every harmful paired-valid start,

so no unsafe execution reached the contingency stage. The median counterfactual change in maximum loading was 3.879 percentage points, with a network-cluster bootstrap 95% confidence interval of 0.261 to 10.744. All 53 paired states and all eight network medians were positive.

The AC-OPF campaign produced 25 paired-valid outcomes from 35 attempts over five networks. PCC stopped all 25 consequential transformations before solver start. The median relative objective effect was 5.96%, with a network-cluster bootstrap 95% confidence interval of 2.61 to 14.07%, and the maximum observed effect was 27.12%. All five network medians were positive.

The attempt accounting is part of the result. Three AC N-1 attempts and ten AC-OPF attempts did not produce paired-valid numerical effects, but they were retained in the experiment record rather than removed from the denominator. This separates two questions that are often conflated in conversion studies. Solver availability asks whether both paired studies can be solved. Operational consequence asks how much the answer changes when both studies are available. PCC prevention is evaluated at the release boundary, so the gate stops the harmful transformed target before either question can become an unsafe protected-solver start.

Figure 2 reports the attempt accounting, network-level effects, and prevention counts for both AC campaigns. The paired-valid design separates transformation-induced task changes from ordinary solver failures. The reported effects therefore come from cases where both the source and transformed studies could be solved.

5.2. DC-SCOPF

In DC-SCOPF, the main failure mode was false security. Across 50 operating states and five networks, the study evaluated 12,340 candidate-outage rows. The paired analysis identified 369 false-secure dispatches: the transformed problem appeared secure, but the intended contingency model violated its limit. PCC blocked all 369 before computation, and no harmful solver start occurred. The one-sided 95% upper bound on the harmful-start rate was 0.81%.

These false-secure cases were not marginal. The median post-contingency loading excess was 0.241 p.u., the median hidden load shedding was 5.20 MW, and the median relative cost understatement was 1.04%. All five network medians were positive for all three effects. The largest case, case500, contributed 5,820 rows and 85 false-secure dispatches.

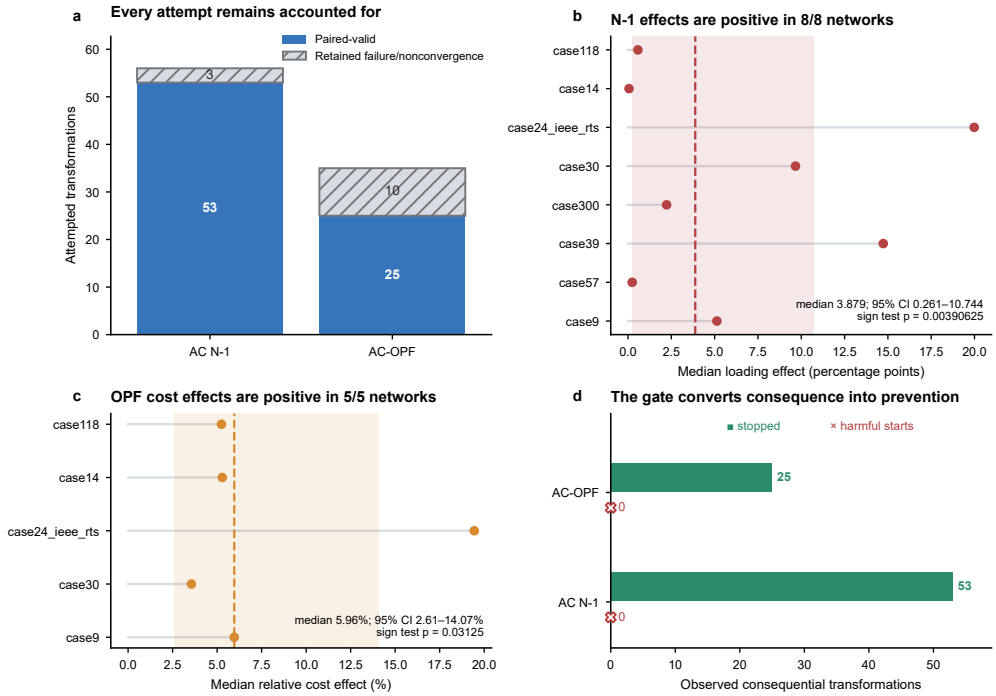

Figure 2: Operational consequences and prevention in the AC studies. PCC prevented every consequential AC N-1 and AC-OPF solver start, with consistent physical and economic effects across networks.

The mechanism is straightforward. A transformed task can remove or misdirect the outage that should reveal a binding constraint. The optimization then finds a dispatch that appears feasible for the transformed contingency set. When the same dispatch is checked against the intended contingency, the hidden overload appears. Load shedding and cost understatement measure the operational consequence of this mismatch from two directions: the first quantifies the emergency action needed to restore feasibility, and the second quantifies how much cheaper the misleading transformed problem looked before the intended constraint was restored.

The 369 false-secure rows show that task-semantic corruption changes both feasibility and economics. The transformed problem can suppress the contingency that carries the binding post-contingency constraint, so the dispatch appears secure at a lower cost. The intended contingency then exposes the physical overload and the associated corrective requirement. PCC blocks this pathway before the optimization result is treated as actionable. The

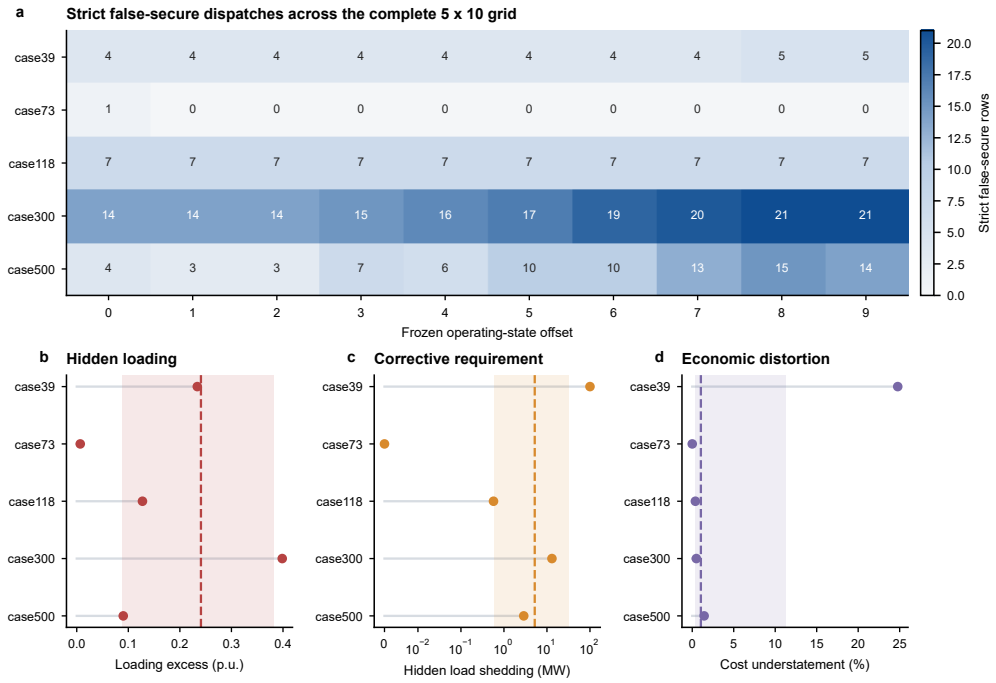

Figure 3: DC-SCOPF outcome atlas. False-secure dispatches span the five-network by ten-state grid, with network-level effects on loading, load shedding, and operating cost.

result is a prevention claim rather than a detection-after-failure claim: every strict false-secure dispatch in the 50-state campaign was stopped before protected solver execution.

Table 1 reports the operational consequence and prevention counts for the AC N-1, AC-OPF, and DC-SCOPF campaigns.

Table 1: Operational consequence and prevention.

Task	Valid/ attempted	Blocked	Harmful starts	Median effect	95% CI	p
AC N-1	53/56	53	0	3.879 pp	0.261–10.744 pp	0.0039
AC-OPF	25/35	25	0	5.96%	2.61–14.07%	0.0313
DC-SCOPF	50/50 states	369	0	0.241 p.u.	0.088–0.382 p.u.	0.0313

Figure 3 shows the full 5×10 state grid and the network-level outcome heterogeneity.

5.3. Semantic baseline ladder

The ablation ladder shows which task evidence prevented consequential solver starts. Across 1,320 harmful transformations and 660 lawful controls, B0 and B1 accepted all harmful cases, B2 accepted 880, B3 accepted 440, B4 accepted 220, and B5 accepted none. Every baseline accepted all 660 lawful transformations. The full PCC therefore produced an observed harmful-release rate of 0/1,320 and a lawful-retention rate of 660/660. The one-sided 95% exact upper bound on the harmful-release probability was 0.2267%.

The ladder was monotone by construction. Signing alone did not establish task semantics, global identity did not establish task coverage, task coverage did not establish attribute preservation, and attribute equality did not bind authoritative relation and intervention evidence. The ordered baselines therefore isolate the value of each obligation in the PCC release decision.

The accepted harmful counts also identify the missing obligation at each level. B0 and B1 accepted every harmful case because structural availability and artifact signing do not inspect task semantics. B2 removed the cases that violated global identity but still accepted task-footprint, attribute, and intervention failures. B3 removed missing task-footprint cases but still accepted transformations with incorrect attributes or relation semantics. B4 removed attribute drift but still accepted failures that required authoritative trace and intervention binding. Full PCC removed the remaining class because it checked the relation evidence that connects a declared task action to the transformed operational object.

Table 2 summarizes the ordered semantic baseline ladder.

Table 2: Ordered semantic baseline ladder.

Baseline	Lawful acc.	Harmful acc.	Harmful rate
`B0_structural_only`	660/660	1320/1320	1.000
`B1_signed_artifact_v1`	660/660	1320/1320	1.000
`B2_global_identity`	660/660	880/1320	0.667
`B3_task_footprint`	660/660	440/1320	0.333
`B4_attribute_invariants`	660/660	220/1320	0.167
`B5_full_PCC_v2`	660/660	0/1320	0.000

Figure 4 shows the monotonic reduction in harmful acceptance. Each

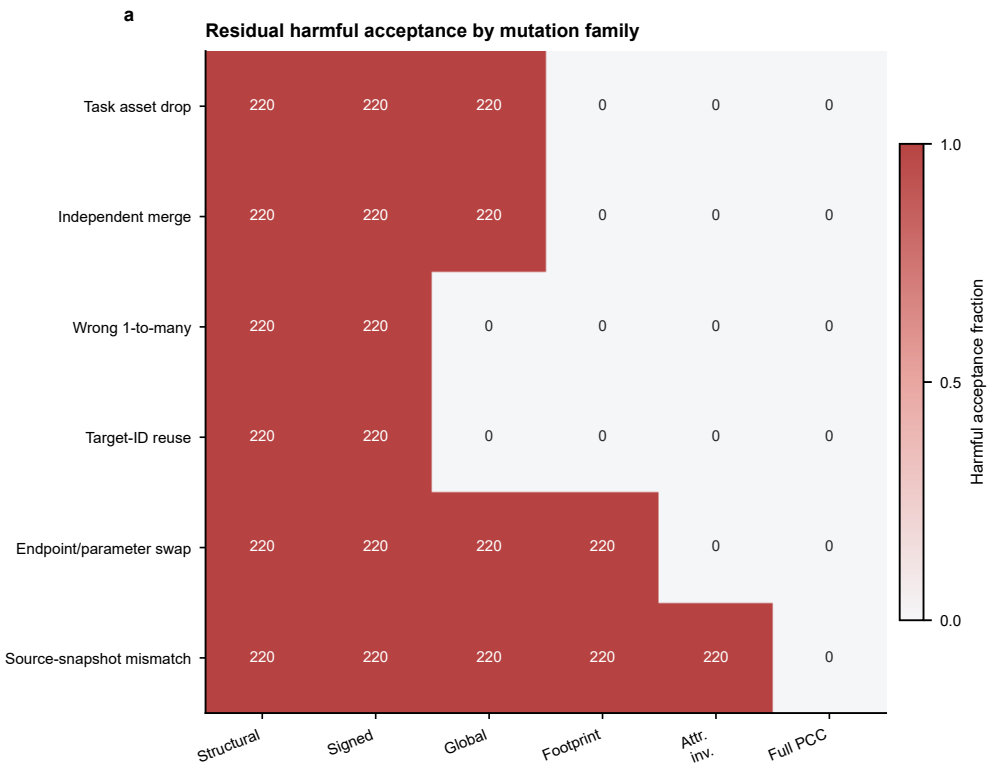

Figure 4: Protection against controlled task-semantic corruption. Full PCC eliminates harmful acceptance while preserving every lawful transformation.

added semantic obligation removed a distinct failure family, while lawful acceptance remained unchanged.

5.4. Decision reason taxonomy and runtime scaling

The release gate also returned repair information at case-study scale. It produced machine-readable reasons for 13,660 of 14,447 evaluated records across 11 reason families. The most frequent DC-SCOPF reasons were `task_selector_not_preserved` and `task_target_missing`, which identify the model elements required for repair. The semantic campaign returned 220 unresolved decisions for missing task-asset mappings, while the external roundtrip returned none.

Verification remained subsecond over the tested range. The p95 latency increased from 1.80 ms at 118 assets to 215.09 ms at 13,659 assets. These timings include the verification gate, not the downstream computation avoided

when a target was rejected.

The reason taxonomy turns prevention into an operational workflow. In the semantic campaign, unresolved decisions came from missing task-asset mappings. In the external roundtrip, lawful transformations carried complete evidence and produced no unresolved outcomes. PCC keeps the launch closed and gives the converter or model engineer the specific missing relation, attribute, or selector to repair before reissuing the certificate.

5.5. Structural standards, solver reproduction, and holdout controls

The standards and portability checks gave the same distinction between structural conformance and task preservation. The official Svedala APL control conformed to the selected APL 1.1.1 SHACL shapes. PCC additionally detected missing authoritative identity evidence for one of eight task assets in the same target. On the untouched PowSyBl bundle, PCC accepted the complete task proof and pypowsybl imported the 59-element network successfully. QoCDC 4.1.4 evaluation covered 15 Level 1 to Level 4 checks.

The independent solver comparison reproduced the aligned formulation across numerical stacks. The initial PowerModels DCP run matched all nine statuses but only 3/8 mutually optimal objectives because it omitted transformer tap and phase-shift terms. With the transformer-aware DCMP formulation, the two stacks agreed on 9/9 statuses and all eight mutually optimal objective and generation values.

This control strengthens the portability result by separating formulation alignment from task preservation. The DCP/DCMP contrast isolates the transformer-treatment effect: once transformer terms were aligned, PYPOWER and PowerModels agreed to numerical tolerance. PCC then verifies a different property on top of the aligned computation, namely whether the transformed task carries the relation and intervention evidence needed for the solver call.

The untouched CGMES holdout extended the result to a separate source corpus. PCC accepted the complete proof over eight BaseVoltage task assets and rejected the target after one proof relation was removed. The blind external-tool challenge accepted 127/127 lawful roundtrips, accepted no harmful task transformations, and produced no harmful solver starts.

Table 3 separates the standards conformance checks from the untouched-source and independent-stack controls.

Table 3: Standards and untouched-source controls.

Artifact	Check	Outcome	Results/ elements	Interpretation
Official Svedala APL control	APL 1.1.1 SHACL	True	0	Structural conformance control
QoCDC development control	QoCDC 4.1.4 subset L1-L4	True	15	Applicable-subset control
Untouched PowSyBl bundle	APL 1.1.1 merged-graph run	False	765	Task-semantic and import holdout
Untouched PowSyBl bundle	pypowsybl 1.15.0 import	success	59	59 reported elements across six tables

6. Discussion

The main implication of the results is that model validation should be tied to the intended operational task. A converted network can pass syntax checks, import into a solver, and converge while still changing the N-1, OPF, or SCOPF problem. PCC addresses this gap at the point where the transformed model is released for computation. The release decision is therefore made on task evidence, not on file validity alone.

This distinction explains the behavior of the semantic baseline ladder. Artifact signatures and global identifiers preserve provenance and object continuity, but they do not prove that the solver receives the same study. Task coverage improves the decision, and attribute checks remove another class of errors. The remaining failures disappear only when relation, trace, and intervention evidence are included. This progression shows that task preservation is a joint property of the converted assets, their parameters, and the operations applied to them.

The operational studies show why this distinction matters for power-system analysis. In the AC cases, corrupted transformations changed contingency loading and OPF objective values even when both paired studies were solvable. In the DC-SCOPF cases, the transformed model produced false-secure dispatches that hid overloads, load shedding, and cost understatement. These are not bookkeeping errors in the exchange format. They are changes in the operating conclusion drawn from the model.

PCC complements existing structural validation rather than replacing it. SHACL, QoCDC, and solver import checks remain useful for detecting malformed or unsupported model data. PCC adds a task layer that asks whether the specific buses, branches, generators, limits, costs, and interventions used by the study have survived conversion. The Svedala, PowSyBl, CGMES

holdout, external roundtrip, and independent-solver checks show that this layer can be placed around realistic model and computation paths.

The innovation lies in enforcing task semantics at the moment of computation. Existing validation layers can certify an exchange profile, an import route, a signed artifact, or a solver formulation. PCC connects those layers to the operational question being asked. The same method covers N-1 loading, AC-OPF cost, and DC-SCOPF false security because the contract is expressed in terms of selected assets, required attributes, relation evidence, and interventions. This abstraction gives a single release mechanism for studies that otherwise use different solvers and result metrics.

Earlier contract work provides useful reference points. Design-by-contract methods specify preconditions, postconditions, and invariants for software components [14], and proof-carrying authorization attaches logical evidence to access decisions [15]. Contract-based design and assume-guarantee methods use contracts for component composition, refinement, and system-level design reasoning [16, 17]. PCC brings this checkable-obligation view to a different boundary. It releases a converted grid model only when the evidence links the solver task to the selected equipment, attributes, relations, and interventions.

The comparison also clarifies the innovation. Traditional contracts describe software behavior or component assumptions. Provenance records describe how an artifact was derived. Structural validators describe whether a data object satisfies a schema or profile. PCC combines these ideas at an execution boundary and expresses the obligations in power-system semantics. The result is a task-bound proof that the selected grid objects, numerical attributes, and solver interventions support the specific study about to run. This explains why the same structurally valid CGMES artifact can be accepted for one declared task, left unresolved for another, and rejected for a third.

The experiments also show the value of task-bound evidence over whole-model equality. Whole-model equality is poorly matched to legitimate conversion, normalization, and solver-specific reduction because local identifiers, file organization, and inactive objects can change. Whole-file availability and signature checks operate at the opposite extreme and can release an artifact that changed the study. PCC uses the operational level: it tolerates irrelevant representational differences and requires evidence for the assets and actions that define the requested computation.

The design maps naturally to deployment. PCC is strongest for sched-

uled contingency screening, OPF studies, SCOPF campaigns, and automated conversion pipelines where the selected assets and solver actions are known before execution. The converter emits or preserves authoritative relation evidence, issuer keys identify approved preparation paths, and solver access is routed through the release gate. These engineering choices make the experimental prevention behavior transferable to practical model-transformation workflows.

The evaluation focuses on public networks, selected CGMES material, and three high-value study families: AC N-1, AC-OPF, and DC-SCOPF. The corruption scenarios target task-semantic failures that directly affect solver conclusions, including missing assets, incorrect relations, target reuse, endpoint drift, and source-snapshot mismatch. This scope supports the central contribution: PCC makes task-preserving evidence auditable and enforceable for declared power-system computations, and the evaluated campaigns show that this enforcement prevents consequential transformed-task errors from reaching the solver.

The computation cost was small compared with the studies it protects. Verification stayed below 216 ms at 13,659 assets, so the gate can be applied before repeated contingency analysis or optimization runs. The method is most directly applicable when the operational task can be declared before solver execution and the converter can emit relation evidence. Under that condition, PCC provides a practical route from model conversion to task-valid power-system computation.

The broader value is reproducible model governance for solver-facing studies. PCC records which transformed objects were trusted, why they were trusted, which obligations were checked, and whether execution was released. That record is independent of the solver implementation and remains readable after the numerical run. It gives operators, researchers, and software maintainers a common object for reviewing conversion evidence, reproducing a release decision, and improving converter traces. In this sense, PCC turns transformation correctness into a concrete artifact rather than an implicit assumption hidden inside the modelling pipeline.

7. Conclusion

This paper introduced proof-carrying contracts for task-preserving power-system model transformation. PCC links source and transformed snapshots, relation evidence, required attributes, and intervention semantics to the

solver-release decision. A converted network is released for N-1, OPF, or SCOPF analysis only after the declared task is preserved.

Across the evaluated cases, this task layer changed which transformed models reached computation. PCC rejected all 1,320 harmful transformations and accepted all 660 lawful controls. It prevented every consequential AC N-1 and AC-OPF solver start and blocked 369 false-secure DC-SCOPF dispatches across 50 operating states. The hidden effects involved branch loading, operating cost, and load shedding, showing that task-semantic preservation changes power-system conclusions rather than only model metadata.

The standards, holdout, scaling, and independent-solver checks place PCC alongside existing model-validation and computation tools. When the task is declared and relation evidence is available, task preservation becomes an explicit execution rule for model-driven power-system analysis.

Data and code availability

The data supporting this study and the code used to generate the results are publicly available in the project GitHub repository and the archived Zenodo release at `https://doi.org/10.5281/zenodo.21796488`. The release contains the implementation, frozen protocols, corpus manifests, hashes, evidence schemas, experiment runners, machine-generated summaries, manuscript figures, figure source data, and machine-generated tables. Third-party public datasets and standards artifacts are referenced by source URL, release or commit, licence record, and SHA-256 rather than relicensed.

Author contributions

Zixuan Liu: conceptualization, methodology, software, formal analysis, investigation, visualization, writing - original draft. Wei Xiong: conceptualization, validation, supervision, writing - review and editing, project administration.

Conflict of interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported in part by the National Natural Science Foundation of China (62202148). It was also supported by the Natural Science Foundation of Hubei Province (2022CFB908 and 2019CFB530). It was also supported by the China Scholarship Council (201808420418).

Acknowledgements

The authors thank the open benchmark and standards communities whose public materials supported the study.

References

[1] S. M. S. Carvalho et al., A scalable interior-point framework for stochastic n-1 secure pre-dispatch in large-scale power systems, Electric Power Systems Research (2026). https://doi.org/10.1016/j.epsr.2026.113452.

[2] J. Zhang et al., GAT-quantile-MLP-based surrogate model for joint chance-constrained OPF with topology and renewable uncertainties, Electric Power Systems Research (2026). https://doi.org/10.1016/j.epsr.2026.112979.

[3] S. Feng et al., Power system fault diagnosis and cascading failure early warning method based on rule knowledge embedding, Electric Power Systems Research (2026). https://doi.org/10.1016/j.epsr.2026.113016.

[4] J. D. Lara, C. Barrows, D. Thom, D. Krishnamurthy, and D. Callaway, PowerSystems.jl, a power system data management package for large scale modeling, SoftwareX 15 (2021) 100747. https://doi.org/10.1016/j.softx.2021.100747.

[5] M. Larhrib, M. Escribano, C. Cerrada, and J. J. Escribano, Converting OCL and CGMES rules to SHACL in smart grids, IEEE Access 8 (2020) 177255–177266. https://doi.org/10.1109/ACCESS.2020.3026941.

[6] M. Larhrib, M. Escribano, R. Cerrada, and J. J. Escribano, An ontological behavioral modeling approach with SHACL, SPARQL, and RDF applied to smart grids, IEEE Access 12 (2024) 82041–82056. https://doi.org/10.1109/ACCESS.2024.3412656.

[7] X. Li, X. Zuo, R. Liu, Y. Lu, C. Li, and S. Lin, SHACL-based validation method of knowledge graph for power system model, Electric Power 55 (2022) 119–125, 228. https://doi.org/10.11930/j.issn.1004-9649.202107093.

[8] World Wide Web Consortium, Common Grid Model Exchange Standard (CGMES) library, official ENTSO-E overview page (2026). https://www.entsoe.eu/data/cim/cim-for-grid-models-exchange/.

[9] ENTSO-E, Common Grid Model Exchange Standard (CGMES), official ENTSO-E standards and interoperability page (2026). https://www.entsoe.eu/major-projects/common-information-model-cim/cim-for-grid-models-exchange/standards/Pages/default.aspx.

[10] World Wide Web Consortium, Shapes Constraint Language (SHACL) (2017). https://www.w3.org/TR/shacl/.

[11] World Wide Web Consortium, SHACL Advanced Features (2017). https://www.w3.org/TR/shacl-af/.

[12] ENTSO-E, Common information model (cim) for grid models exchange, official ENTSO-E overview page (2026). https://www.entsoe.eu/digital/common-information-model/cim-for-grid-models-exchange/.

[13] G. C. Necula, Proof-carrying code, Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (1997) 106–119. https://doi.org/10.1145/263699.263712.

[14] B. Meyer, Applying design by contract, Computer 25 (10) (1992) 40–51. https://doi.org/10.1109/2.161279.

[15] A. W. Appel and E. W. Felten, Proof-carrying authentication, Proceedings of the 6th ACM Conference on Computer and Communications Security (1999) 52–62. https://doi.org/10.1145/319709.319718.

[16] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, Taming Dr. Frankenstein: contract-based design for cyber-physical systems, European Journal of Control 18 (3) (2012) 217–238. https://doi.org/10.3166/ejc.18.217-238.

[17] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen, Contracts for system design, Foundations and Trends in Electronic Design Automation 12 (2–3) (2018) 124–400. https://doi.org/10.1561/1000000053.

[18] World Wide Web Consortium, PROV-DM: The PROV Data Model (2013). https://www.w3.org/TR/prov-dm/.

[19] World Wide Web Consortium, PROV-Overview: An Overview of the PROV Family of Documents (2013). https://www.w3.org/TR/prov-overview/.

[20] World Wide Web Consortium, PROV-N: The Provenance Notation (2013). https://www.w3.org/TR/prov-n/.